KHU Unreal Engine Tutor (KUT)

정답 대신 질문과 힌트로 학생의 생각을 이끌고 고치고 확인하는 일은 학생에게 남기는 언리얼 에디터 안의 AI 학습 튜터

Unreal Engine 5.8 에디터 플러그인입니다. 학교가 제공하는 ChatKHU(Claude Sonnet 5.5)로 답하고 도움의 양은 학생이 L0~L3 중에서 고르며 코드와 에셋은 바꾸지 않습니다.

1 팀 소개와 작품 소개

팀명	**KUT** '커트'로 읽음 KHU Unreal Engine Tutor(경희대 언리얼 엔진 튜터)
팀원	**강태원** 경희대학교 소프트웨어융합학과 ktw3389@khu.ac.kr
역할	1인 개발. 문제 정의, 도움 수준과 코칭, 학습 기록 설계, 검수 기준 마련, 화면 검수와 시연
개발 방식	설계와 판단은 직접 하고 엔진 API 확인과 구현, 빌드, 테스트는 AI 코딩 에이전트(Codex, 이후 Claude Code)의 도움을 받음. 학생용 패키지의 모델 호출은 ChatKHU로 보냄
기간	2026. 10. 2. ~ 10. 27.

작품 소개

KUT는 Unreal Engine 5.8 에디터 안에서 쓰는 AI 학습 튜터입니다. 정답을 알려 주지 않고 질문과 힌트로 학생이 원인을 스스로 좁히게 합니다. 게임엔진기초 수업에서 언리얼을 처음 배우며 겪은 어려움에서 출발했습니다. 과제가 막히면 튜터가 학생 프로젝트의 **Blueprint(노드를 이어 만드는 시각적 코드) 그래프와 컴파일 오류, Output Log(엔진 기록), C++ 빌드 오류**를 직접 읽고 그 근거와 살펴볼 곳을 보여 줍니다. 도움의 양은 학생이 **네 단계(L0~L3)** 중에서 고르고 튜터는 코드와 에셋을 바꾸지 않습니다. 고친 뒤 Compile과 Play로 확인하고 그 과정을 자기 말로 기록하는 일은 학생이 합니다.

근거 먼저
답하기 전에 에디터 자료부터 읽고 카드로 보여 줍니다

도움은 고른 만큼
질문 하나(L0)부터 전체 설명(L3)까지 학생이 고릅니다

고치는 건 학생
코드와 에셋은 그대로 두고 고치는 일은 학생이 합니다

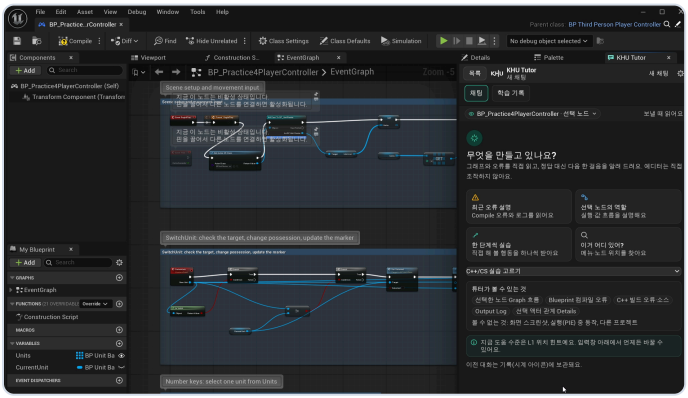


Fig 1. 수업 실습 과제 Practice4를 연 Blueprint 에디터와 오른쪽에 붙인 KUT 패널

2 작품 개요

개발 배경

1. **게임엔진기초 수업.** 언리얼을 처음 배울 때 엔진이 너무 넓어 어디서부터 익혀야 할지 막막했습니다. Window 메뉴에서 창을 여는 항목만 22개입니다(Fig 2). 오류가 나면 에디터는 오류 난 노드나 줄은 알려 주지만 실행선과 참조, 값 가운데 무엇이 틀렸는지 알기 어려웠고 조교가 수강생 모두를 봐 줄 수는 없었습니다(개발자의 수강 경험). 낯선 프로그램을 고치는 개발자도 파일 안팎을 오가는 데만 시간의 평균 35%를 썼습니다[1](입문자 연구는 아님).
2. **AI만으로 코딩하면 남는 것이 없음.** 범용 챗봇의 답은 버전이나 메뉴와 맞지 않을 때가 있고 답을 통째로 주기 쉽습니다. 그 답을 붙여 과제를 끝내면 다음 과제에서 다시 막힙니다. 고교 수학 약 1,000명의 실험에서 GPT-4를 제한 없이 쓴 학생은 AI 없이 본 시험에서 AI를 안 쓴 학생보다 17% 낮았고 교사가 넣어 둔 정답과 힌트로 답 대신 힌트를 주게 설계한 튜터는 그 손실을 대부분 막았습니다[2]. 이 결과를 설계의 근거로 삼았습니다.
3. **업계가 요구하는 기본기.** 넥슨 2026 넥토리얼 모집 요건[4]은 C++ 또는 C# 개발, CS 지식의 코드 구현, 생성형 AI로 코드를 쓰고 검증하는 능력을 요구합니다. 크래프톤 정글의 학습 원칙[6](묻기 전에 스스로 답 찾기)을 설계 모티브로 삼았습니다.

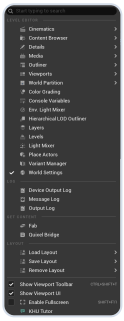


Fig 2. Level Editor의 Window 메뉴 (실제 화면)

문제	그래서 세운 원칙
어디에서 무엇이 틀렸는지 알기 어렵고 챗봇의 답은 버전이나 메뉴와 맞지 않을 때가 있다	→ **원칙 1** 살펴볼 곳은 내 프로젝트의 근거로 좁혀 준다
AI가 답을 통째로 주면 배운 것이 남지 않는다[2]	→ **원칙 2** 정답 대신 질문과 힌트로 생각을 이끌고 고치고 확인하는 일은 학생이 한다. 절차는 학생이 L3를 고를 때만 설명하고 코드는 바꾸지 않는다

범위 엔진 전체를 먼저 다 배울 수는 없으니 지금 막힌 과제에서 볼 곳을 좁히는 데서 시작한다. 학습 순서 전체를 짜 주는 일은 하지 않는다.

목표 시스템

- **근거로 좁히기(원칙 1)** 그래프와 오류, 로그, 허용된 소스를 읽어 근거와 볼 곳을 보여 주고 다음에는 학생이 직접 보게 합니다.
- **도움의 양 조절(원칙 2)** L0 질문 하나부터 L3 전체 단계까지 학생이 고르고 튜터는 코드와 에셋을 바꾸지 않습니다.
- **수행과 설명(원칙 2)** 학생이 직접 고치고 Compile과 Play로 확인한 뒤, 판단한 이유를 자기 말로 기록합니다.
- **C++/CS로 넓히기** 넥슨 2026 넥토리얼 모집 요건[4]을 참고해, Blueprint로는 다루기 어려운 C++/CS 실습 12과제를 더했습니다.

1 관찰과 도움 선택	2 읽고 답하기	3 근거와 위치 확인	4 고치고 시험	5 재확인과 설명	6 학습 기록
학생	**튜터**	**학생**	**학생**	**튜터와 학생**	**학생**
기대한 동작과 본 것을 적고 L0~L3 중 도움을 고름	허용된 자료를 먼저 읽고 고른 수준만큼만 답함	'확인한 근거'를 보고 노드 링크로 위치를 엶	직접 고치고 Compile과 Play로 확인	튜터가 고친 곳을 다시 읽고 학생은 결과와 판단을 설명	실패한 시도와 내가 한 일, AI가 한 일까지 남김

Fig 3. KUT로 과제 하나를 푸는 순서. 답이 부족하거나 결과가 예상과 다르면 1로 돌아가 같은 대화에서 다시 묻습니다.

③ 작품 구성 및 상세 내용

Window ▸ KHU Tutor 메뉴나 툴바 버튼으로 여는 도킹 패널입니다. 위쪽에서 **채팅**과 **학습 기록**을 오가고 왼쪽에 채팅 목록, 가운데에 대화와 근거 카드, 아래에 입력창과 도움 수준, 도우미가 있습니다. 설정 화면에서는 ChatKHU 키를 등록하고 연결을 확인합니다. 키는 ●로 가려 입력하고 이 PC에 DPAPI로 암호화해 저장합니다.

근거 카드와 노드 링크

튜터는 답하기 전에 허용된 도구로 자료를 읽고 무엇을 읽었는지 카드로 남깁니다. Blueprint는 알아보기 어려운 스크린샷 대신 노드를 한 줄씩 적은 텍스트(**Graph Digest**)로 읽어 노드 번호와 연결 상태를 그대로 받고 답변의 파란 **#번호**를 누르면 에디터가 그 노드로 이동해 선택합니다.

```
#42 [Call] Possess | exec>#23 | InPawn<#3
#2 [Call] K2_SetRelativeRotation | exec>none |
Target=!UNCONNECTED | ERR "This blueprint (self) is not a
SceneComponent, therefore ' Target ' must have a connection."
```

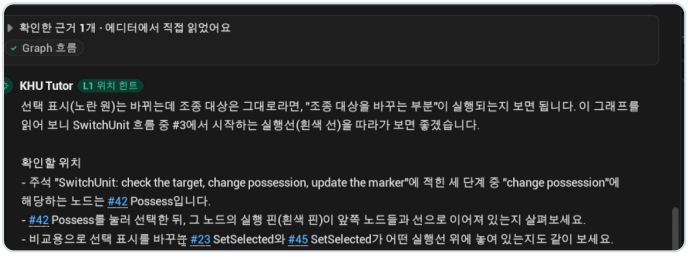


Fig 4. '확인한 근거' 카드(Graph 흐름)와 L1 위치 힌트 답변. #42를 누르면 Possess 노드로 이동합니다(실제 응답).

네 가지 도움 수준

L0	L1	L2	L3
질문으로 생각하기	**위치 힌트**	**부분 단계**	**전체 단계**
답 대신 확인할 질문 하나	살펴볼 위치만	핵심 한두 단계와 이유	이유를 포함한 전체 설명

기본은 L1입니다. L0에서는 **위치 이동 도구와 노드 링크, 근거 상세를 코드에서 막고**, 답의 내용이 단계를 지키는지는 지침과 응답 검수로 확인합니다. 단계를 나눠 학생이 고르게 한 것은 처음 설계(10. 2.) 때 정했고 제출을 준비하며 찾은 메타분석(연구 164편)도 같은 방향입니다. 도움 없이 혼자 발견하게 하면 학습에 이롭지 않았고 피드백과 예시, 단계적 도움, 설명 유도를 더하면 이로웠습니다[5]. 단계마다 돕는 튜터링 시스템이 사람 튜터에 가까운 효과를 냈다는 연구 종합[3]도 그렇습니다. L0는 질문 전에 자기 답부터 찾아보게 하는 크래프톤 정글의 학습 원칙[6]과도 방향이 같습니다. 합성 시험에서 L0에 막힌 학생 역할에게 질문만 이어져 '모르겠어요'가 두 번 이어지면 L1로 올릴지 묻는 카드를 더했습니다. 자동으로 올리지는 않습니다.

학습 코칭

필요할 때 답변 아래에 **'질문 돌아보기'** 카드가 붙습니다. 학생이 쓴 말을 인용해 무엇을 보고 판단했는지, 도구로 확인한 결과인지 학생이 알려 준 내용인지를 나눠 보여 주는 메모로, 점수는 매기지 않습니다. 다음 행동은 학생이 고르고 자동 코칭은 끌 수 있습니다. 듣기만 할 때보다 스스로 설명을 만들어 낼 때 더 많이 배운다는 연구 종합(ICAP)[7]과 같은 방향입니다.

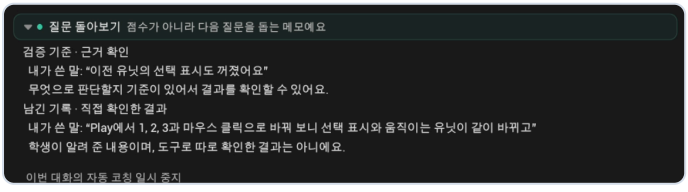


Fig 5. 학생이 알린 Play 결과를 '학생이 알려 준 내용'으로 나눠 보여 주는 질문 돌아보기 카드(같은 시연의 실제 화면)

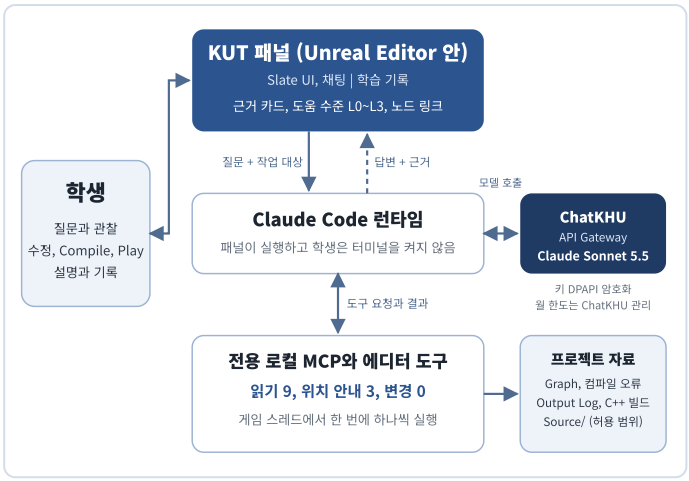


Fig 6. 시스템 구성. 튜터 도구는 에디터 읽기 9개(연결 확인 포함), 위치 안내 3개, 학습 기록 조회 2개입니다. Claude Code는 기본 도구를 모두 끈 채(`--tools ""`) 튜터 도구만 허용하며 실행 때 받은 도구 목록이 튜터 도구와 다르면 답변을 실패로 처리합니다. 바꾸는 도구가 없는 것은 설계상의 제한이며 사용 전후 파일은 비교하지 않았습니다.

C++/CS 실습 12과제

1. Blueprint에서 C++ 흐름 읽기
2. 아이템 자료구조 고르기
3. 조건에 맞는 대상 탐색
4. AI가 준 C++ 코드 검토
5. 기능을 컴포넌트로 조합하기
6. 내 프로젝트에서 검증하기

기본 6과제와, 조건을 바꿔 다시 풀어 보는 6과제

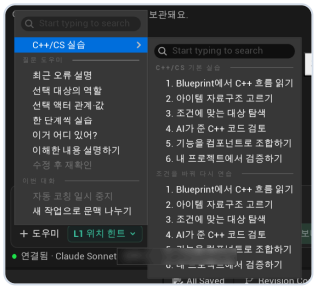


Fig 7. 도우미 메뉴의 C++/CS 실습(실제 화면)

과제를 고르면 목표와 문제, 내 역할, 검증 기준이 입력창에 채워지고 작성과 빌드, 경계값 확인은 학생이 합니다. 4번은 AI가 준 **First(Values)** 함수의 입력 가정을 학생이 찾습니다. Blueprint만으로 C++ 실력을 판단하지 않습니다.

학습 기록

1. 참고할 대화 → **2. 내 설명** → **3. 저장과 공유**

시도와 실패, 판단한 이유, 확인한 방법과 결과, 내가 한 일과 AI가 한 일을 자기 말로 남기고 사실과 기여, 개인정보를 확인한 뒤 '기록 복사'로 저장합니다. 기록 속 AI 답변에는 '사실 확인 필요' 표시가 붙고 '설명 다듬기'는 짧은 질문과 참고 자료 카드만 준비할 뿐 보내는 것은 학생입니다.

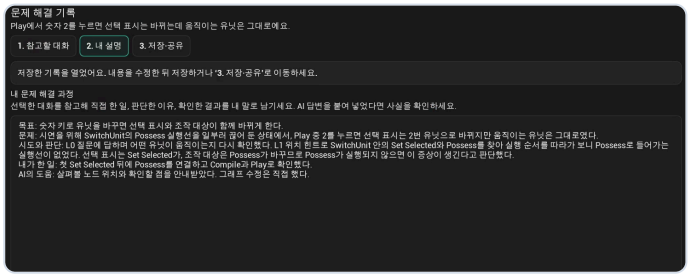


Fig 8. 학습 기록 2단계 '내 설명'. 문제, 시도와 판단, 내가 한 일, AI의 도움을 학생의 말로 적습니다(실제 화면, 입력 문장은 예시).

시중 제품과의 차별점 공식 문서 기준으로 Ultimate Engine CoPilot은 자산 생성·수정과 Learn 강의, Blueprint Analyst는 노드 설명, GraphDeck은 Explain·lesson과 AI 그래프 생성·배포, Epic AI Assistant는 답변과 C++ 생성을 합니다[8]. 답을 보류하는지, 드러낼 양을 학생이 고르는지, 수정 도구가 있는지로 보면 대신 만들어 주는 쪽입니다. KUT는 과제 상태를 읽은 뒤에도 답을 주지 않고 질문과 힌트로 생각을 이끕니다. 드러낼 양은 학생이 L0~L3로 고르고 L0에서는 링크까지 코드로 막으며 바꾸는 도구가 없습니다.

4 개발 세부 내용

구현과 안전, 비용

- **구조.** 센서와 읽기 9개·위치 안내 3개 도구의 KHUTutorTools, 에이전트 루프·모델 연결·UI의 KHUTutor 두 모듈입니다.
- **실행 경로.** 패널이 띄운 **Claude Code**가 **ChatKHU**로 모델을 부르고 튜터 도구는 이 PC 안의 전용 MCP로 받아 화이트리스트와 스키마 검사 뒤 게임 스레드에서 하나씩 실행합니다.
- **원칙과 안전.** 도움 수준마다 허용 도구와 표시를 바꾸고 요청마다 L0 규칙을 다시 붙입니다. API 키는 **Windows DPAPI**로 암호화하고 설정이 범위 밖이면 유료 요청을 막으며 학생 패키지는 작업당 **240크레딧**에서 묻고 **800**에서 멈춥니다.

실패에서 고친 설계

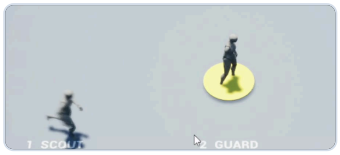
응답 검수 → 원인 분류 → 지침과 코드 수정 → 다시 검수

점검 기준 개발 중 답변은 개발자와 AI 검토 에이전트, 자동 규칙 검사가 AI 튜터 평가 틀[9]에 해당하는 기준(위치 짚기, 답 바로 주지 않기, 다음 행동, 일관성)과 도움 수준 규칙으로 점검했고 실패한 응답은 남겼습니다.

1. **근거를 읽지 않음.** 실전 프로젝트에서 Haiku 5.5의 L1 답변이 자료를 읽지 않고 학생에게 찾게 해서 "도움은 적게 줘도 근거는 먼저 읽는다"를 지침에 넣었습니다(실전 재시험은 남음).
2. **질문이 답을 암시함.** L0 질문에 고칠 위치가 들어가 L0를 "지금 아는 것으로 답할 관찰 하나"로 바꿨습니다.
3. **기록에 내부 지시문이 보임.** 화면용 질문과 모델용 자료를 나눴습니다.

5 구현 결과

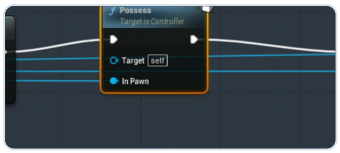
이번 단계의 질문 근거를 읽고 볼 곳을 안내하면, 실제 수업 과제에서 학생이 직접 고치고 확인하는 데까지 이어지는가? 학생이 더 잘 배우는지는 다음 단계의 질문입니다(아래 평가 계획).



1 2를 누르면 노란 원은 GUARD로 가는데 움직이는 건 SCOUT
과제 완성본에서 Possess 실행선을 일부러 끊어 둠



2 L0로 세 번 묻다 L1로 바꾸자 #42 링크가 Possess를 가리킴. 실행 핀 양쪽이 비어 있음



3 학생 역할을 맡은 개발자가 Set Selected, Possess, Set Selected 순서로 다시 연결



4 Play에서 1, 2, 3 키와 마우스 클릭으로 바꾸자 노란 원과 움직이는 유닛이 함께 바뀜

Fig 9. 수업 실습 과제 Practice4 시연(10. 27. 녹화). 고친 뒤 '수정했어요, 재확인'으로 튜터가 새 그래프를 다시 읽어 이전 근거와 비교했고 학생이 '해결됐어요'라고 알린 뒤 같은 시연에서 질문 돌아보기와 학습 기록 1단계까지 진행했습니다. 답변은 실제 응답이며 10. 18. ChatKHU 크레딧이 바닥난 뒤라 개발용 Claude Code 구독 연결로 받았습니다.

확인한 것	판정 기준과 근거	판정	결과	확인 범위
빌드와 기능 회귀	검사 코드에 정해 둔 기대 결과와 같은가(UE Automation 테스트)	자동	**검사 모두 통과**	유료 모델이나 개발용 연결이 필요한 검사는 생략
배포 후보	공식 패키징(BuildPlugin) 성공, 패키지 파일 검사, 빈 Blueprint 프로젝트에서 로드	자동	**패키지 검사·로드 통과**	Win64, 화면과 모델 호출 없이 로드만
실제 과제 적용	핵심 Blueprint 컴파일 오류 0, 원본 파일이 바뀌지 않았는가(해시 비교)	자동	**핵심 Blueprint 컴파일, 원본 보존**	원본이 아닌 사본
흐름 시연	과제가 요구한 동작(숫자 키로 선택 표시와 조작 대상이 함께 바뀜)이 끊은 연결을 고친 뒤 돌아오는가	Play에서 관찰	**동작함**	과제 1개를 1회, 학생 역할은 개발자, 답변은 개발용 연결
L0 질문 규칙 (개발 중 점검)	초보자가 답할 질문 하나인가, 답을 암시하지 않는가, 이미 말한 정보를 다시 묻지 않는가. 기준은 응답 전에 적어 두고 튜터에는 보내지 않음(사전 등록[10]과 같은 방식)	AI 검토 에이전트 (개발용)	**13개 중 2개 위반, 수정 뒤 같은 두 대화 8개 충족**	합성 대화 4개(프로젝트 읽기 없음), 대부분 L0, 개발용 연결. 규칙 준수만 점검

Table 1. 무엇을 어떤 기준으로 누가 판정했는지(2026. 10. 26. 기준, 시연은 10. 27.). 학생의 학습 효과는 아직 재지 않아 이 표에 없습니다.

6 기대 효과

결론. 위치를 알던 개발자가 학생 역할을 맡아 완성본에 일부러 끊어 둔 연결 한 곳을 KUT의 안내대로 고치고 Play로 확인했습니다. 이 시연에서 질문부터 근거, 직접 수정, 확인까지 이어졌고 학습 기록은 1단계까지 진행했습니다. 처음 보는 학생이 맞는 곳을 찾는지, 더 잘 배우는지는 아직 모릅니다.

기대 효과(아래 계획으로 확인할 가설) 막힌 지점을 현상으로 좁혀 묻는 습관, 근거를 보고 직접 고쳐 확인하는 습관이 생기고 다시 읽을 기록이 남으며 에디터 안에서 질문하게 되리라 봅니다. 연구윤리 심의와 동의를 거쳐 성적과 분리해 확인합니다.

무엇을	도구와 근거	판정
사용성	SUS 10문항[11]	수강생 본인
작업 부담	NASA-TLX(정신적 부담, 좌절감 등 6개 척도)[12]	수강생 본인
사용 중 문제	입문자 4~5명 사용성 시험(인원 근거[13]), 생각을 말하며 쓰기	관찰자 기록
볼 곳 정확도	처음 보는 고장 사례를 미리 정해, 튜터가 가리킨 노드를 정답 노드와 대조	자동(학생 없이 가능)
답변 품질	AI 튜터 평가 틀 8개 항목[9]과 도움 수준(L0~L3) 준수, 일치도 κ[14]. 같은 연구에서 LLM 평가자 2종의 판정이 사람과 맞지 않아 사람이 판정[9]	개발자가 아닌 평가자 2명
질문 습관	질문 기록을 미리 정한 기준(관찰한 현상 포함 여부 등)으로 분류, 일치도 κ[14]	평가자 2명
학습 효과	사전 검사 뒤 KUT, 범용 챗봇, AI 없음 세 조건으로 나눠 과제를 하고 AI 없이 비슷한 오류의 위치를 찾아 고치게 함(정확도, 시간)[2]. 1~2주 뒤 재검사	조건을 모르는 채점자

한계 학생의 학습 효과는 아직 측정하지 않았습니다(학생 배포 전). 처음 보는 사례에서 튜터가 맞는 곳을 가리키는지도 재지 않았고 틀린 곳을 가리킬 수 있어 근거 카드로 학생이 확인하게 했습니다. 문제 제기는 개발자의 경험과 당시 기록, 공개 자료에 근거했고 수강생 설문은 하지 않았습니다. 원래 학생 개인의 ChatKHU 크레딧으로 쓰도록 설계했지만 크레딧이 넉넉하지 않아 폭넓게 쓰기 어렵습니다. 앞으로는 학생이 쓰는 Codex나 Claude Code 같은 CLI를 연결하거나(지금은 Claude Code 연결만 개발용으로 시험) 크레딧을 늘려야 합니다. 10. 18. 크레딧이 바닥난 뒤의 검수와 시연 답변은 개발용 연결로 받았고 학생 패키지의 ChatKHU 경로로 다시 해 보지는 않았습니다. 다음 버전에는 실행 중 값·Debugger 센서, 승인·되돌리기를 갖춘 자동 수정(L4), Claude Code 동봉 배포를 계획합니다.

참고문헌 근거 논문은 그 학술지나 학회를 다루는 공개 등급(CCF, CORE, JUFO 등)에서 모두 최상위인 것만 골랐습니다. 처음 설계(10. 2.)는 개발자의 판단이고 개발 중에 본 자료는 [2][4][8], 설계 모티브는 [6]입니다. 나머지는 제출을 준비하며 대조했고 [9][11][12][14]는 평가 도구의 원 출처, [4][6][8]은 공개 자료입니다.

1. Ko 외 (2006). IEEE TSE 32(12).
2. Bastani 외 (2025). PNAS 122(26).
3. VanLehn (2011). Educ. Psychol. 46(4).
4. NEXON. 넥토리얼 2026 모집공고.
5. Alfieri 외 (2011). J. Educ. Psychol. 103(1).
6. KRAFTON. 크래프톤 정글 소개 (2023).
7. Chi & Wylie (2014). Educ. Psychol. 49(4).
8. Epic AI Assistant, Blueprint Analyst, CoPilot, GraphDeck 문서 (2026. 10. 확인)
9. Maurya 외 (2025). NAACL.
10. Nosek 외 (2018). PNAS 115(11).
11. Brooke (1996). SUS.
12. Hart & Staveland (1988). NASA-TLX.
13. Nielsen & Landauer (1993). CHI.
14. Cohen (1960). EPM 20(1).

GitHub https://github.com/tw-kang01/KUT
소스, 설치 안내, 테스트 자료